

NEON DRIFT — Cyberpunk Snake Game

Development Report

Project: A browser-based Snake game with a futuristic cyberpunk / "Tokyo Drift" aesthetic — neon lighting, glowing trails, an animated grid, and explosion effects.

Built with: AI-assisted development (Claude), using iterative prompting to design, build, and debug a single self-contained HTML/CSS/JavaScript file.

Published game link: <https://claude.ai/public/artifacts/7ea024ad-bd75-4d05-82ac-5a8e3685d660>

1. Overview

The goal was to build a complete, playable Snake game that didn't just work mechanically but carried a strong visual identity: a dark neon cityscape feel, a glowing snake, and arcade-style juice (screen shake, particles, sound). The game was built conversationally — describing what was wanted, reviewing the result, and then refining through follow-up prompts.

2. Prompt Iterations Used During Development

The project moved through four main prompting stages.

Iteration 1 — Concept and vibe. The first prompt described the core idea: a Snake game that "should look futuristic in a cyberpunk style, with neon lights, giving a Tokyo Drift kind of vibe." This was deliberately high-level — it set the aesthetic direction before any code existed. The response was a plan covering gameplay, look and feel, and extra features, which was approved before building began.

Iteration 2 — Build the game. After approving the plan, a simple confirmation ("yes") triggered the full build. This produced the first working version: snake mechanics, neon gradient trail, animated grid, pulsing food, glitch-styled title and game-over screens, score and best tracking, keyboard plus swipe controls, and pause.

Iteration 3 — Add a feature (the "boom"). The next prompt requested a specific effect: when the snake hits the target, it should produce "a small boom, like a bomb boom." This was a focused, additive request layered onto the existing game rather than a rebuild.

Iteration 4 — Bug fixing. A prompt reported that the game showed errors and wouldn't play. This shifted the conversation from building to debugging — identifying the cause and patching it.

3. What Worked Well

Several things went smoothly:

- **Leading with the vibe, not the code.** Describing the aesthetic ("cyberpunk," "neon," "Tokyo Drift") up front gave the build a clear creative direction. The result captured the intended mood on the first try — the gradient trail, glow, and animated grid all landed.
 - **Approving a plan before building.** Reviewing a written plan first meant the actual build matched expectations, avoiding a large rewrite.
 - **Incremental, single-feature requests.** Asking for the explosion effect as its own step kept the change contained and easy to review, rather than bundling many changes at once.
 - **Self-contained output.** Keeping everything in one HTML file (no external assets, sound synthesized in-browser) made the game easy to run, share, and publish.
 - **Plain-language feature requests.** Describing the explosion as "a bomb boom" in everyday words was enough — there was no need to specify the technical implementation (screen shake, shockwave ring, sound synthesis); those were inferred.
-

4. What Did Not Work

- **A hidden ordering bug in the first build.** The game's "best score" variable was referenced before it was properly set up in the code. JavaScript ran that line first, threw an error, and the entire script stopped — meaning the game wouldn't start at all. This wasn't visible in the plan or in a quick read of the code; it only surfaced on running the game.

- **The error wasn't obvious from the surface.** The game *looked* finished, which made the failure confusing at first — nothing in the visible design hinted at the problem. It took actually loading the game to discover it.
 - **Audio timing quirk.** Browsers block sound until the user interacts with the page, so the explosion sound couldn't play on load. This is expected browser behavior rather than a true bug, but it had to be accounted for (the START button doubles as the user interaction that unlocks audio).
-

5. How Prompts Were Improved Over Time

The prompting style evolved as the project progressed:

- **From broad to specific.** Early prompts set overall direction ("make it cyberpunk"). Later ones narrowed to single, well-defined features ("add a boom when the snake eats the target"). Narrow prompts produced more predictable, easier-to-review changes.
 - **Describing the *experience*, not the *implementation*.** Instead of asking for technical components, the requests described what the player should feel or see — "a bomb boom." This let the desired outcome drive the technical choices.
 - **Reporting symptoms clearly during debugging.** The most useful debugging prompt simply stated the observable problem ("there are errors and I can't play"). Describing the symptom — rather than guessing at the cause — was enough to locate and fix the issue.
 - **Building in reviewable steps.** Each prompt produced a checkpoint that could be tested before moving on, so problems could be caught against a known-good baseline rather than tangled into a large batch of changes.
-

6. Challenges Faced During the Process

- **Invisible runtime errors.** The biggest challenge was a bug that didn't show up until the game was actually run. A finished-looking product can still contain a single line that halts everything — a reminder that testing the live result matters as much as reviewing the code.
- **Balancing visual richness with playability.** The game layers several effects at once — animated grid, glowing trail, pulsing food, particles, shockwaves, screen shake. These had to coexist without overwhelming the core gameplay or hurting performance.

- **Cross-device input.** Supporting both desktop (arrow keys / WASD) and mobile (swipe) controls meant handling two different input styles in one codebase.
 - **Browser audio restrictions.** Working within the browser rule that audio can't autoplay required routing the first sound through a user action (the START button).
 - **Tuning "game feel."** Effects like explosion strength, screen-shake intensity, and the snake's acceleration are subjective — getting them to feel punchy but not annoying is a matter of judgment rather than a single correct value.
-

7. Summary

The project showed that a strong result comes from a clear creative brief, building in reviewable increments, and describing desired *outcomes* rather than dictating implementation. The main lesson learned was the value of testing the running product, not just the code — the one significant bug was invisible until the game was actually played, and was resolved quickly once the symptom was reported plainly.

Game built as a single self-contained HTML file. The public link is in Section 0 above; anyone with the URL can play the game in their browser.